Multi-Provider LLM Agent with Per-User System Prompt Customization

Requirement Document: User Story, Impact Analysis, Technical Specifications and Size Estimation

Scope of this document: frontend, API, backend, database, validation and testing tasks for both phases of the requirement.

Prompt used for the Requirement

Act as a Senior Business Analyst. Generate a detailed, professional Agile user story for the requirement "Multi-Provider LLM Agent with Bring-Your-Own-Key Model Selection and Per-User System Prompt Patching."

TITLE

Multi-Provider LLM Agent with Bring-Your-Own-Key Model Selection and Per-User System Prompt Patching

As a user of the agent, **I want** to select an LLM provider and model, supply my own API key, and tailor the agent's system prompt to my own rules and output format, **so that** every task I run uses my own provider quota and produces output that meets my business requirement without affecting any other user.

DESCRIPTION

The requirement is delivered in two phases.

Phase 1: Multi-Provider Model Selection with User-Supplied API Key. The agent supports multiple LLM providers, including OpenAI, Anthropic (Claude), Google (Gemini) and Mistral. When a user selects a provider, the system retrieves the list of available models directly from that provider's API. The user selects a model and supplies a personal API key for that provider. All agent tasks executed by the user are then sent to the selected provider and model using the user's key, so usage is charged against the user's own quota and rate limits.

Phase 2: Per-User Dynamic System Prompt. The agent runs on a base system prompt maintained by the platform. Each user can view the effective system prompt, modify its rules, output format and behavioural instructions from the frontend, preview the change, and apply it. Applying a change stores a user-specific patch on top of the base prompt. The patched prompt is used only for that user's executions. For example, User 1 may be satisfied with the base output while User 2 applies a patch to change the output format; User 3 and User 4 may apply different patches of their own. Each user's execution remains fully independent.

ACCEPTANCE CRITERIA

Phase 1

- The list of supported providers is displayed to the user.
- On provider selection, available models are fetched live from the provider's API using the user's key and displayed.
- The user can save, replace and delete an API key per provider.
- The API key is validated against the provider before it is saved; an invalid key is rejected with a clear message.

- The API key is stored encrypted, is never returned to the frontend in full (only the last four characters), and never appears in logs.
- Every agent execution uses the user's selected provider, model and key; no platform key is used as a fallback.
- Provider errors (invalid key, quota exceeded, rate limit, model unavailable) are mapped to clear, user-facing messages.

Phase 2

- The user can view the effective system prompt (base prompt plus the user's active patch).
- The user can edit rules, output format and behaviour sections and preview the difference before applying.
- Applying a change creates a new patch version for that user only; other users' prompts are unaffected.
- The user can view patch history, revert to any earlier version, or reset to the base prompt.
- Protected sections of the base prompt (safety and compliance guardrails) cannot be removed or overridden by a patch.
- If the base prompt is updated by the platform, each user's patch is re-applied; any patch that no longer applies cleanly is flagged to the user and the base prompt is used until resolved.
- Each execution records the base prompt version and patch version used, for traceability.

EXPECTED RESULT

Users can run the agent against the LLM provider and model of their choice using their own API key, with all usage billed to their own provider account. Each user can tailor the system prompt to their own requirement through the frontend, and the patched prompt is applied only to that user's executions, while the base prompt and its guardrails remain intact for all users.

TASKS

- Review provider APIs (model listing, authentication, error codes) and implement one adapter per provider.
- Implement secure API key storage, validation and masking, and the provider/model/key screens.
- Define the base prompt structure; implement patch versioning, revert, reset and the prompt editor.
- Integrate prompt resolution and user key routing into the agent execution path.
- Execute unit, integration, isolation, security and regression testing; raise and retest defects.

Prompt for Impact Analysis

Act as a Senior Business Analyst and Solution Architect. Perform an impact analysis for introducing multi-provider model selection with user-supplied API keys and per-user system prompt patching into the existing agent. Analyze the impact on the frontend, API layer, backend services, database, validation rules and testing, and identify risks, dependencies and assumptions.

IMPACT ANALYSIS SCOPE

- **Agent execution path:** currently bound to a single provider and a platform key; must resolve provider, model, key and prompt per user at run time.
- **Authentication and authorization:** all new resources (keys, preferences, patches) are user-owned and must be scoped to the authenticated user.
- **Security and compliance:** introduction of user secrets requires encryption at rest, masking and log redaction.

- **Prompt management:** the base prompt moves from a static configuration to a versioned, structured resource.
- **Audit and cost:** execution records capture provider, model and prompt versions without secrets; LLM usage moves from platform-billed to user-billed.
- **Regression:** existing agent behaviour for users who have not customised their prompt must remain unchanged.

Technical Specifications

Act as a Senior Technical Business Analyst, Solution Architect and Full-Stack Developer with expertise in LLM provider integration, secure secret management, REST API design and relational data modelling. Provide the technical specification for the requirement with clear separation of concerns across frontend, API, backend, database, validation and testing.

1. FRONTEND

Responsibility: user interaction only. The frontend holds no provider logic, never stores an API key locally, and never calls an LLM provider directly.

- **Provider and Model Selection:** dropdown of supported providers; model list loaded from the API after selection, with loading, empty and error states.
- **API Key Management:** masked input field; save, replace and delete actions; status indicator (Valid, Invalid, Not Set) and last four characters only.
- **System Prompt Editor:** section-based editor (Rules, Output Format, Behaviour); protected sections shown read-only; side-by-side diff preview before applying.
- **Apply, Revert and Reset:** Apply creates a new patch; history panel lists versions with timestamps; Revert and Reset to Base require confirmation.
- **Execution Panel:** shows the active provider, model and prompt version for each run; provider errors displayed as clear, generic messages.

2. API LAYER

Responsibility: request contract, authentication, authorization and response format. All endpoints operate on the authenticated user’s own data.

Method	Endpoint	Purpose
GET	/api/v1/providers	List supported providers
GET	/api/v1/providers/{providerId}/models	Fetch available models for the provider using the user's key
PUT	/api/v1/users/me/credentials/{providerId}	Validate and save or replace the user's API key
DELETE	/api/v1/users/me/credentials/{providerId}	Delete the user's API key
PUT	/api/v1/users/me/model-preference	Save the selected provider and model
GET	/api/v1/users/me/system-prompt	Return the effective prompt and active patch version
POST	/api/v1/users/me/system-prompt/preview	Return the diff and resolved prompt without saving
POST	/api/v1/users/me/system-prompt/patches	Apply a new patch (creates a new version)
GET	/api/v1/users/me/system-prompt/patches	List patch history
POST	/api/v1/users/me/system-prompt/patches/{version}/revert	Revert to an earlier version
DELETE	/api/v1/users/me/system-prompt/patches	Reset to the base prompt
POST	/api/v1/agent/execute	Run a task with the user's provider, model, key and prompt

- Status codes: 200/201, 400 validation, 401/403 access, 404, 409 patch conflict, 422 invalid key, 429 rate limit, 502 provider unavailable. Error body: `code`, `message`, `correlationId`, with generic messages only.
- API keys are accepted only in request bodies over HTTPS and are never echoed in any response.

3. BACKEND SERVICES

Responsibility: business logic, provider integration, secret handling and prompt resolution.

- **Provider Adapter Layer:** a common interface (`listModels`, `validateKey`, `execute`) with one adapter per provider. Adapters normalise request and response formats and map provider errors to platform error codes. Adding a new provider requires only a new adapter and a configuration entry.
- **Model Catalogue Service:** calls the provider's model listing endpoint; caches results per provider for a short period (for example, 15 minutes) to limit provider calls.
- **Credential Service:** encrypts keys using envelope encryption with a managed key service (KMS) before storage; decrypts only in memory at execution time; redacts keys from logs and error traces.
- **Prompt Service:** loads the current base prompt; applies the user's active patch at execution time; enforces protected sections; produces previews and diffs; manages versions, revert and reset.
- **Execution Orchestrator:** resolves the user's model preference, key and effective prompt, calls the provider adapter, and records an audit entry with provider, model, base prompt version, patch version, token usage and status.

4. DATABASE

Responsibility: persistence, data integrity and user-level isolation.

Table	Key Columns	Notes
providers	id, code, name, base_url, is_active	Reference data for supported providers
user_provider_credentials	id, user_id, provider_id, encrypted_key, key_last4, status, validated_at	Unique (user_id, provider_id); key stored encrypted only
user_model_preferences	user_id, provider_id, model_id, updated_at	One active preference per user
system_prompt_base	id, version, content, protected_sections, created_at, is_current	Versioned base prompt; one current version
user_prompt_patches	id, user_id, version, base_version, patch_content, is_active, created_at	Unique (user_id, version); one active patch per user
execution_audit	id, user_id, provider_id, model_id, base_version, patch_version, tokens_in, tokens_out, status, created_at	No keys or prompt content stored

- Foreign keys to user and provider tables; indexes on `user_id` for all user-owned tables. Patches are append-only; revert creates a new version copying an earlier one.

5. VALIDATION

Responsibility: input correctness, security and business-rule enforcement at the API and service layers.

- **API key:** non-empty, provider-specific format check, then a live validation call to the provider before saving.
- **Model selection:** the model must exist in the provider's current model list and the user must hold a valid key for that provider.
- **Patch content:** maximum size limit; only editable sections may be changed; protected sections must remain unchanged; the resolved prompt must stay within the selected model's context limit.
- **Prompt content safety:** patches are checked for instructions that attempt to disable guardrails or extract secrets; such patches are rejected.
- **Authorization:** every request is checked so that a user can read or modify only their own keys, preferences and patches.
- **Execution:** execution is blocked with a clear message if no valid key or model preference exists.

6. TESTING

Responsibility: verification of functionality, isolation, security and regression.

- **Unit testing:** provider adapters (mocked responses and error mapping), patch apply and revert logic, prompt resolution, encryption and masking.
- **Integration testing:** model listing, key validation and execution against each provider's sandbox or test account.
- **API contract testing:** request and response schemas, status codes and error format for every endpoint.

- **User isolation testing:** User 1, User 2, User 3 and User 4 apply different patches; each execution uses only that user's prompt and key.
- **Security testing:** keys never returned in responses or written to logs; cross-user access attempts are rejected; protected sections cannot be overridden.
- **Regression testing:** users without a patch receive output equivalent to the current agent behaviour.
- **User acceptance testing:** end-to-end scenarios for provider selection, key setup, prompt customisation and execution, with evidence recorded and defects raised and retested.

End-to-End Technical Flow

Phase 1: Provider and Key Setup

User selects provider → Frontend → `PUT /credentials/{providerId}` → Validation (format and live key check) → Credential Service encrypts and stores key → `GET /providers/{id}/models` → Model Catalogue Service fetches model list → User selects model → Preference saved

Phase 2: Prompt Customisation

User edits prompt sections → Preview (diff returned) → Apply → Validation (protected sections, size, safety) → Prompt Service stores new patch version for the user only

Execution

User submits task → `POST /agent/execute` → Execution Orchestrator resolves model preference, decrypts the user's key, builds base prompt plus the user's active patch → Provider Adapter calls the selected model → Response returned to user → Audit record stored

Prompt for Size Estimation

For the above requirement, provide the requirement breakdown with the size estimation for each technical component, along with the overall story-point estimate and the rationale.

REQUIREMENT BREAKDOWN WITH SIZE ESTIMATION

Requirement: Multi-Provider LLM Agent with Per-User System Prompt Customization

Overall proposed size: 2 stories, 34 Story Points (Phase 1: 18 SP, Phase 2: 16 SP)

ID	Requirement / Task	Technical Component	Phase	Size (SP)
REQ-01	Provider adapter interface and adapters for four providers	Backend	1	5
REQ-02	Credential storage, encryption, validation and masking	Backend / Database	1	3
REQ-03	Provider, model and credential endpoints	API	1	2
REQ-04	Provider/model selection and API key screens	Frontend	1	3
REQ-05	Execution routing with user key and error mapping	Backend	1	2
REQ-06	Phase 1 integration, security and regression testing	Testing	1	3
REQ-07	Structured base prompt and versioned patch data model	Database	2	2
REQ-08	Patch apply, preview, revert and reset logic with protected sections	Backend / Validation	2	5
REQ-09	System prompt endpoints	API	2	2
REQ-10	Prompt editor with diff preview, apply and history	Frontend	2	3
REQ-11	Phase 2 isolation, security and regression testing	Testing	2	4
	Total			**34 SP**

ESTIMATION RATIONALE

- **Provider adapters, 5 SP:** four provider APIs with different authentication, model listing and error formats.
- **Credentials, 3 SP:** KMS encryption, live validation and log redaction.
- **Patch logic, 5 SP:** section-level patching, protected sections and conflict handling on base prompt updates.
- **Frontend, 3 SP each phase:** dynamic model loading and key states; section editor, diff view and history.
- **API, execution routing and data model, 2 SP each:** standard endpoints and a contained change to the existing execution path.
- **Testing, 3 SP and 4 SP:** per-provider integration and key security; multi-user isolation, guardrail bypass and regression.

RISKS, ASSUMPTIONS AND DEPENDENCIES

- **Risks:** provider API changes (mitigated by adapters and live model listing); guardrail removal through patches (mitigated by protected sections and content validation).
- **Assumptions:** each user holds a valid provider account and key; existing authentication supplies a stable user identifier.
- **Dependencies:** a managed key service for encryption; sandbox or test accounts for each provider.